CST8219 – C++ Programming

Lab3: Object-Oriented Student System and Employee Inheritance System

Marks: 20
Course Weight: 6%

OBJECTIVE

This lab extends Lab 2 by first converting the student system into an object-oriented design, then expanding the project into an employee inheritance system. Students must demonstrate encapsulation, constructors, deep copy, move semantics, virtual functions, friend functions, abstract classes, polymorphism, and safe memory handling.

By completing this lab, students will be able to:

- Encapsulate data using classes and private members**.**
- Implement multiple constructor types, copy/move operations, and destructors.
- Use a virtual member function for runtime polymorphism.
- Use a friend function to support controlled access for comparison or display.
- Build an abstract base class and derived classes.
- Manage collections of objects safely using smart pointers.
- Validate all interactive input safely.

Pre-Requisites:

- Successful completion of **Lab 2**
- This lab must be completed by extending Lab 2 project.
- Existing Lab 2 functionality should remain available where applicable, but the design must be upgraded to object-oriented classes and inheritance.

Part 1: Student OOP Refactor

The first part converts the Lab 2 student system into a class-based design. The student record logic remains the same, but it must now be encapsulated inside a Student class with proper constructors, assignment operators, and member functions.

Student Data Model:

Each student object must contain:

- ID: positive integer.
- Name: non-empty string.
- Marks in 3 subjects stored dynamically.
- Total marks.
- Average marks.

- Pass/fail status using enum Status.
- Letter grade using enum class Grade.
- The class must keep all data members private, and the subject marks must be managed safely using smart pointers.

Required enums

```
enum Status { FAIL, PASS };
enum class Grade { A, B, C, D, F };
```

Required Student Class Features:

Create a Student class with:

- Default constructor.
- Parameterized constructor.
- Copy constructor.
- Move constructor.
- Initializer-list constructor.
- Delegating constructor.
- Copy assignment operator.
- Move assignment operator.
- Virtual destructor.
- Virtual display() function.
- Overloaded updateMarks() functions.
- Recursive helper function for total mark calculation.
- **Friend function** for comparing two student objects or displaying a controlled summary.
  Suggested friend function:

  ```
  friend bool compareAverage(const Student& lhs, const Student& rhs);
  ```

  This friend function can return whether one student has a higher average than another, or it can be used to support highest-score logic without exposing private data directly.

Part 2: Employee Inheritance Layer

The second part replaces or extends the student model with an employee hierarchy. This layer introduces inheritance, abstract base classes, overridden functions, and polymorphic behavior.

Employee Data Model:

Each employee object must contain:

- Name.
- Base salary.
- Role-specific fields depending on the derived class.

Derived classes must include:

- SalariedEmployee.
- HourlyEmployee.
- CommissionEmployee.

Suggested derived data:

- SalariedEmployee: bonus.
- HourlyEmployee: hours worked, hourly rate.
- CommissionEmployee: sales, commission rate.

Abstract base class

Create an Employee base class with:

- Private attributes for name and base salary.
- Pure virtual calculatePay() const.
- Virtual displayInfo() const.
- Virtual destructor.

Required virtual usage

At least one **virtual** function must be implemented and used for runtime polymorphism. The menu and employee display logic must call this function through a base-class pointer or reference.

Suggested friend function

A friend function may also be used in employee design, such as:

```
friend bool comparePay(const Employee& lhs, const Employee& rhs);
```

This can be used to find the highest-paid employee while keeping the pay calculation data private.

Menu Requirements

Stage 1 menu

1. Add student records.
2. Display all students.
3. Find highest scoring student.
4. Display class summary.
5. Proceed to Stage 2.

Stage 2 menu

1. Add employee.
2. Display all employees.
3. Find highest-paid employee.
4. Exit.

The key rule is:

- Stage 1 must be completed before Stage 2 begins.
- Stage 2 must reuse the same project skeleton and coding style.
- The student system and employee system should be separate stages, not mixed into one menu.

INPUT VALIDATION RULES:

The program must validate:

- Positive numeric IDs.
- Non-empty names.
- Marks between 0 and 100.
- Non-negative pay-related values.
- Valid menu choices.
- Safe handling of empty collections and missing records.

EDGE CASES TO HANDLE:

- No students or employees exist: show a safe message.
- Copying objects: must produce independent data.
- Moving objects: source must remain valid.
- Self-assignment: handle safely.
- Multiple additions: no memory leaks.
- Highest-score or highest-pay searches with no data: handle gracefully.

Class and File Structure:

Recommended file structure:

- main.cpp
- Student.h
- Student.cpp
- Employee.h
- Employee.cpp
- SalariedEmployee.h
- SalariedEmployee.h/.cpp
- HourlyEmployee.h/.cpp
- CommissionEmployee.h/.cpp
- CMakeLists.txt
- .git

Git Commit Message

```
git commit -m "Completed Combined Lab 3 and 4: OOP, Friend Functions, and
Polymorphism"
```